PROJET DEEP LEARNING

CareerVerse

Recommander un métier IT à un étudiant avec un réseau de neurones : CareerNet

Méthodologie CRISP-DM · Données · Architecture · Résultats · Cas d'utilisation

5 000 étudiants simulés à partir de la base officielle des métiers O*NET
9 métiers IT · 4 000 étudiants pour apprendre · 500 pour régler · 500 pour l'examen final
Résultat : le bon métier est dans le top-3 pour 98,9 % des étudiants du test

Document de support pour le rapport de projet

Sommaire

Le projet en une page	
La méthode CRISP-DM	
Phase 1	Comprendre le besoin
Phase 2	Comprendre les données
Phase 3	Préparer les données
Phase 4	Construire le modèle : architecture CareerNet
Phase 5	Évaluer : les résultats
Phase 6	Déployer : utiliser le modèle
Cas testés : quand le modèle réussit et quand il se trompe	
Limites et améliorations	
Petit dictionnaire des mots techniques	

Comment lire ce document

Chaque partie commence par une explication simple, souvent avec une comparaison de la vie de tous les jours. Les encadrés **« En bref »** résument l'idée à retenir. Les mots techniques sont expliqués dans le dictionnaire à la fin.

Tous les chiffres viennent des expériences lancées sur le découpage final 4 000 / 500 / 500.

Le projet en une page

Le problème. Un étudiant en informatique doit choisir une spécialité : développeur web, cybersécurité, data, cloud… Ce n'est pas facile. Ses notes ne disent pas tout : un très bon étudiant peut être fait pour plusieurs métiers.

L'idée. La plateforme CareerVerse propose des **Career Labs** : des petits exercices pratiques dans chaque domaine (un exercice de sécurité, un exercice web, un exercice base de données…). Elle observe aussi les notes de l'étudiant, ce qu'il pense savoir faire, et ce qui l'intéresse.

La solution. Un réseau de neurones, appelé **CareerNet**, lit toutes ces informations et propose les **3 métiers** qui correspondent le mieux à l'étudiant, avec un pourcentage de confiance pour chacun.

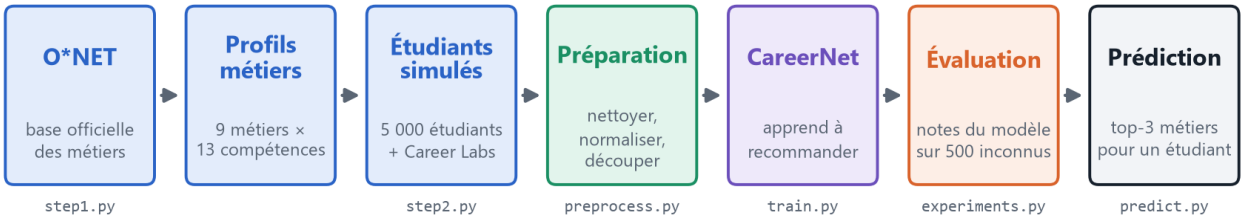


Figure 1. Le chemin complet, des données brutes jusqu'à la recommandation. Sous chaque étape : le programme Python qui la réalise.

Ce qu'on mesure	Résultat	Ce que ça veut dire
Accuracy	90,9 %	Sur 100 étudiants jamais vus, le 1er métier proposé est le bon pour environ 91.
Top-3	98,9 %	Le bon métier est dans les 3 propositions pour presque tous les étudiants.
Macro-F1	0,903	Note moyenne sur les 9 métiers, en donnant la même importance à chacun (sur 1).
Équité	pas d'écart	Le modèle marche aussi bien pour les femmes que pour les hommes.

> **En bref**
> CareerNet regarde le profil de l'étudiant et ses résultats aux exercices pratiques, puis propose 3 métiers. Il trouve le bon métier dans son top-3 presque à chaque fois.

Méthodologie

La méthode CRISP-DM

Pour ne rien oublier dans un projet de données, on suit une méthode connue : **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*). Elle découpe le travail en **6 étapes**.

On peut la comparer à la préparation d'un plat : on décide d'abord quoi cuisiner (le besoin), on regarde les ingrédients qu'on a (les données), on les lave et on les coupe (la préparation), on cuisine (le modèle), on goûte (l'évaluation), puis on sert (le déploiement). Si le plat n'est pas bon, on revient en arrière.

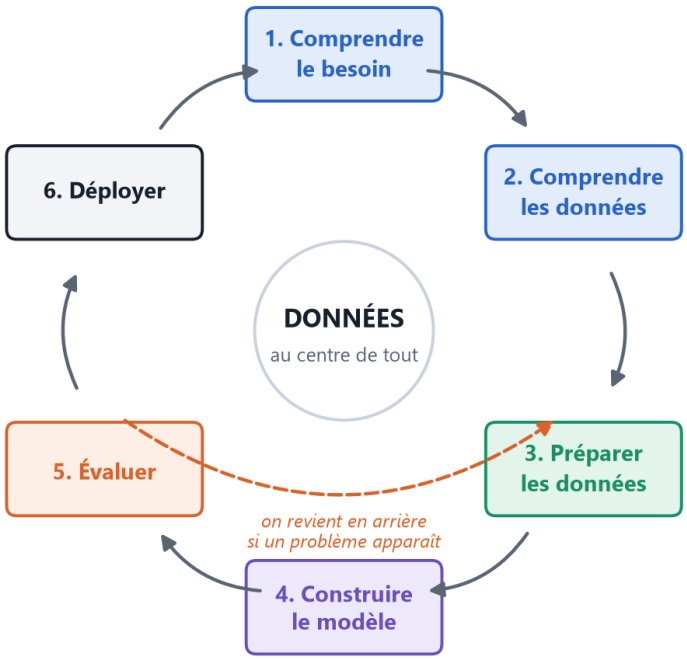


Figure 2. Le cycle CRISP-DM. Les flèches en pointillés montrent qu'on revient en arrière quand un problème apparaît.

Phase	Question posée	Dans ce projet
1. Besoin	Qu'est-ce qu'on veut obtenir ?	Recommander 3 métiers IT à un étudiant
2. Données	Qu'est-ce qu'on a comme informations ?	Base O*NET + 5 000 étudiants simulés
3. Préparation	Comment rendre les données utilisables ?	Nettoyer, mettre à la même échelle, découper
4. Modèle	Quel modèle construire ?	CareerNet : réseau de neurones à 2 branches
5. Évaluation	Est-ce que ça marche vraiment ?	Comparaison avec 4 autres modèles, tests de robustesse et d'équité
6. Déploiement	Comment l'utiliser en vrai ?	Un programme qui reçoit un étudiant et renvoie son top-3

Retours en arrière faits dans ce projet

• En phase 2, on a vu que les profils métiers étaient incohérents (par exemple, un développeur web noté 10/10 en Linux). On a corrigé la façon de les construire.

• En phase 5, on a vu qu'une option (les « labels doux ») n'aidait pas. On l'a abandonnée. On a aussi changé le découpage des données en 4 000 / 500 / 500.

CRISP-DM · Phase 1

Comprendre le besoin

Le modèle doit choisir entre **9 métiers IT**. Chaque métier a une compétence « signature », celle qui le caractérise le plus, et un Career Lab qui lui correspond.

Métier	Ce que fait la personne	Compétence signature	Lab associé
Backend Developer	programme la partie « serveur » des applications	programmation	backend_api
Frontend/Web Developer	crée les pages et interfaces web	web frontend	web_ui
DevOps Engineer	automatise la mise en ligne des logiciels	Docker / Kubernetes	devops_pipeline
Cloud Engineer	gère les serveurs dans le cloud (AWS, Azure…)	cloud	cloud_deploy
Cybersecurity Analyst	protège les systèmes contre les attaques	sécurité	security_ctf
Data Scientist	analyse les données et fait de l'IA	maths / statistiques	data_analysis
Database Administrator	gère les bases de données	SQL / bases de données	sql_database
System/Network Admin	gère les serveurs et le réseau	réseau	network_config
QA/Test Engineer	teste les logiciels pour trouver les bugs	tests / qualité	qa_testing

Comment savoir si le modèle est bon ?

On s'est fixé des critères de réussite **avant** de commencer :

- **Macro-F1** : c'est la note principale. Elle donne le même poids à chaque métier. C'est important car certains métiers sont rares (DevOps) : un modèle qui les ignore aurait quand même une bonne accuracy, mais un mauvais macro-F1.
- **Top-3** : la plateforme montre 3 propositions, donc le bon métier doit être dedans.
- **Robustesse** : le modèle doit marcher même si l'étudiant n'a fait que quelques labs, ou aucun.
- **Équité** : pas de différence de qualité entre femmes et hommes.
- **Faire mieux que les méthodes classiques** (régression logistique, Gradient Boosting). Sinon, utiliser le deep learning ne sert à rien.

> **En bref**
> Le but : choisir le bon métier parmi 9, en étant juste pour tous les métiers (même les rares) et pour tous les étudiants.

CRISP-DM · Phase 2

Comprendre les données

La plateforme n'a pas encore de vrais utilisateurs. On a donc construit les données en **deux étapes**.

Étape A : les profils des métiers (O*NET)

O*NET est une grande base de données publique du ministère du Travail américain. Elle décrit plus de 900 métiers : les compétences demandées, les connaissances, les outils utilisés.

Pour chacun des 9 métiers, on a calculé un **profil de 13 compétences**, chacune notée de 0 à 10 : programmation, Python, Linux, Docker/Kubernetes, cloud, réseau, sécurité, bases de données, web, maths/statistiques, tests, résolution de problèmes, vision système.

- Les notes viennent pour **moitié d'O*NET** et pour **moitié d'un avis d'expert**. O*NET seul donnait des erreurs (un développeur web noté 10/10 en Linux).
- Un contrôle automatique vérifie que chaque métier est 1er ou 2e sur sa compétence signature.
- O*NET n'a pas de données de compétences pour Data Scientist. On a utilisé à la place deux métiers très proches : statisticien et chercheur en informatique.

Étape B : 5 000 étudiants simulés

On a créé 5 000 étudiants « virtuels ». L'idée est la suivante :

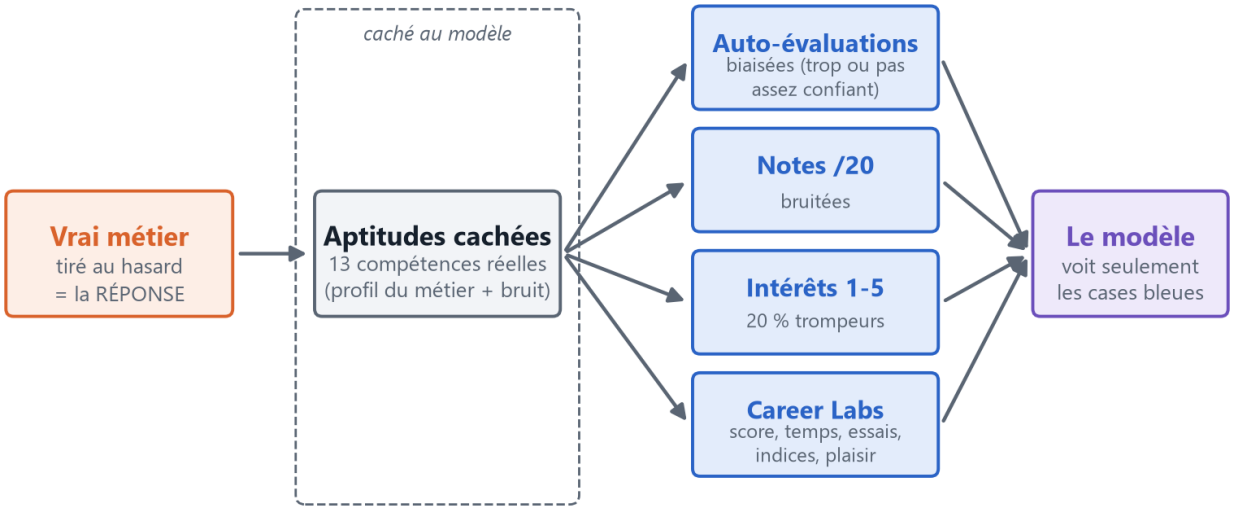


Figure 3. Comment un étudiant est simulé. Le modèle ne voit jamais les aptitudes cachées : il doit retrouver le métier à partir des informations bleues, qui contiennent des erreurs.

1. On tire au hasard le **vrai métier** de l'étudiant. C'est la réponse que le modèle devra trouver.
2. On fabrique ses **vraies aptitudes**, proches du profil de ce métier, avec des différences individuelles. Deux étudiants du même métier ne se ressemblent donc pas parfaitement.
3. On fabrique ce que la plateforme **observe**, avec des imperfections, comme dans la vraie vie : certains étudiants se surestiment, les notes varient, 20 % des étudiants déclarent un intérêt qui ne correspond pas à leur vrai métier.
4. Chaque étudiant ne fait **qu'une partie des labs** : il a plus de chances de faire ceux de son domaine préféré.

Ce que contient le tableau final

Type d'information	Nombre	Échelle	Exemple
Année d'étude et notes (moyenne, maths, programmation, réseaux)	5	1 à 5 ; notes sur 20	maths : 14,5/20
Auto-évaluations : ce que l'étudiant pense savoir faire	13	0 à 10	Python : 7/10
Intérêts déclarés pour 5 domaines	5	1 à 5	sécurité : 5/5
Nombre de labs faits	1	0 à 9	6 labs
Résultats des 9 labs : score, temps, essais, indices, plaisir	45	variées	CTF : 96/100 en 28 min
Le vrai métier (la réponse)	1	9 métiers	Cybersecurity Analyst
Le genre (seulement pour vérifier l'équité)	1	F / M	jamais donné au modèle

Ce que l'exploration des données a montré

Vérification	Résultat
Doublons	0
Valeurs impossibles (ex. note > 20)	0
Informations manquantes (hors labs)	0
Labs non faits	41 % en moyenne : c'est normal, chacun ne fait pas tous les labs
Femmes / hommes	2 486 / 2 514 : équilibré
Métier le plus fréquent / le plus rare	Frontend 718 étudiants, DevOps 240 : 3 fois moins

En bref

Les données imitent de vrais étudiants : des profils métiers réels (O*NET), des informations imparfaites, et des exercices que chacun ne fait pas en entier. Un point à surveiller : DevOps est rare.

CRISP-DM · Phase 3

Préparer les données

Un réseau de neurones ne lit que des nombres, bien rangés et à la même échelle. Cette phase transforme le tableau des étudiants en données prêtes à l'emploi.

1. Garder seulement les bonnes informations

On retire ce qui ne doit pas aider à deviner : l'identifiant de l'étudiant, la réponse elle-même, le genre (pour que le modèle ne puisse pas discriminer) et la colonne « lab réussi », qui répète simplement « score ≥ 50 ».

2. Séparer en deux familles

- **Le profil** : 24 nombres qui décrivent l'étudiant (notes, auto-évaluations, intérêts, nombre de labs).
- **Les labs** : un tableau de 9 lignes (un par lab) et 5 colonnes (score, temps, essais, indices, plaisir).
- **Le masque** : une liste de 9 oui/non qui dit quels labs ont été faits. Un lab non fait n'est jamais remplacé par une note inventée : on dit simplement au modèle de l'ignorer.

3. Mettre tout à la même échelle

Un temps en minutes (de 10 à 150) et un plaisir noté de 1 à 5 n'ont pas la même échelle. Sans correction, le modèle croirait que le temps est plus important juste parce que ses nombres sont plus grands. On transforme donc chaque information pour qu'elle soit centrée autour de 0 (on appelle cela la **normalisation**). Pour les essais et les indices, on applique d'abord un logarithme, qui écrase les valeurs extrêmes.

> **Point important : pas de triche**
> Les réglages de la normalisation (moyennes, écarts-types) sont calculés **uniquement sur les étudiants d'entraînement**. Si on utilisait aussi ceux de l'examen, le modèle aurait un indice sur l'examen à l'avance. On appelle cette erreur une « fuite de données ».

4. Découper en trois groupes

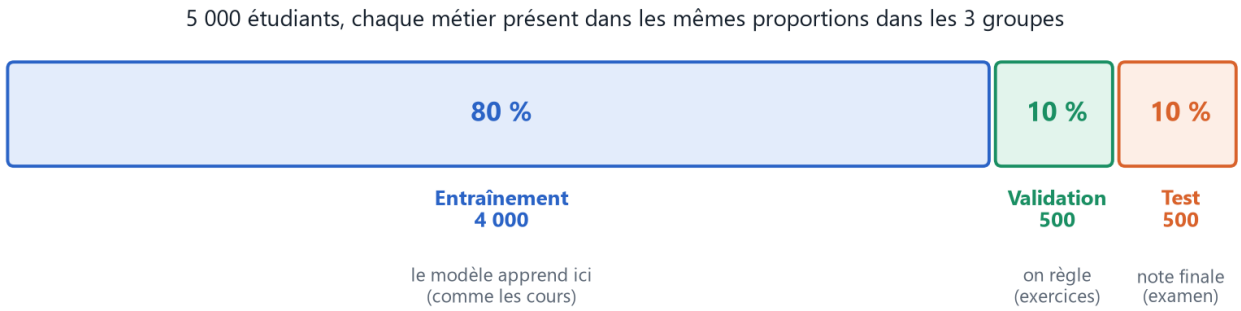


Figure 4. Le découpage des 5 000 étudiants. Comme à l'école : on apprend avec les cours, on s'entraîne sur des exercices, et la note finale vient d'un examen jamais vu.

Le découpage est **stratifié** : chaque métier a la même proportion dans les 3 groupes. Par exemple, il y a environ 5 % de DevOps dans chaque groupe.

5. Compenser les métiers rares

DevOps est 3 fois plus rare que Frontend. Pour que le modèle ne l'oublie pas, une erreur sur un DevOps compte plus fort pendant l'entraînement : **2,31 fois** plus, contre 0,77 pour Frontend. On appelle cela les **poids de classes**.

> **En bref**
> On garde les bonnes informations, on les met à la même échelle sans tricher, on marque les labs non faits au lieu d'inventer des notes, et on découpe en 4 000 / 500 / 500.

CRISP-DM · Phase 4

Construire le modèle : l'architecture CareerNet

C'est quoi un réseau de neurones ?

Un réseau de neurones est une grosse fonction mathématique avec beaucoup de petits réglages, appelés **poids**. CareerNet en a **88 521**. Au début, ces poids sont au hasard et le modèle répond n'importe quoi. Pendant l'entraînement, on lui montre des milliers d'exemples avec la bonne réponse, et il ajuste ses poids petit à petit pour se tromper de moins en moins.

Un **neurone** fait une chose simple : il prend plusieurs nombres, les multiplie chacun par un poids, les additionne, puis applique une petite transformation. En empilant beaucoup de neurones en **couches**, le réseau peut apprendre des règles compliquées.

L'architecture : deux branches qui se rejoignent

CareerNet regarde l'étudiant de deux façons, puis combine les deux avis, comme un jury qui lirait le dossier scolaire d'un côté et regarderait les travaux pratiques de l'autre.

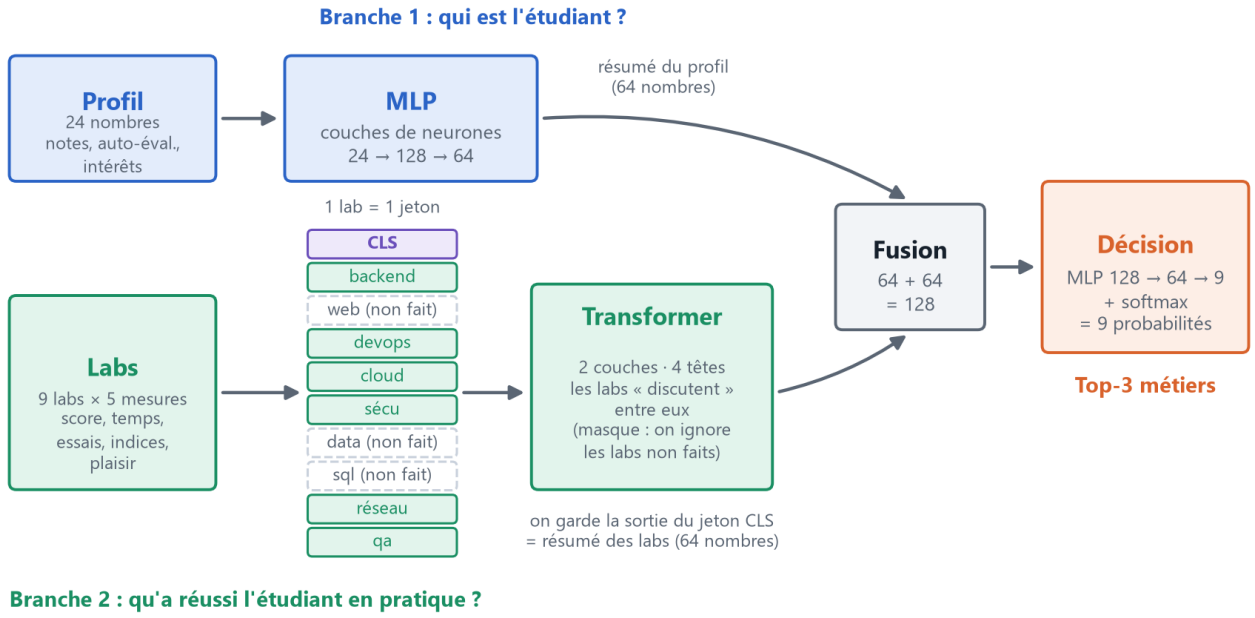


Figure 5. L'architecture de CareerNet. En haut, la branche « profil » ; en bas, la branche « labs ». Les deux résumés sont réunis pour prendre la décision finale.

Branche 1 : le profil (un MLP)

Les 24 nombres du profil passent dans un **MLP** (perceptron multicouche), le type de réseau le plus simple : des couches de neurones les unes après les autres (24 → 128 → 64). À la sortie, on a un **résumé de 64 nombres** qui décrit « qui est l'étudiant ».

Branche 2 : les labs (un Transformer)

C'est la partie la plus originale. Un **Transformer** est le type de réseau utilisé par les grands modèles de langage comme ChatGPT. Ici, au lieu de lire des mots, il lit des labs.

1. **Chaque lab devient un « jeton »**, comme un mot dans une phrase. Ses 5 mesures sont transformées en 64 nombres.

2. **On ajoute à chaque jeton une « étiquette » du lab**. Sans elle, le modèle ne saurait pas si un score de 90 vient du lab sécurité ou du lab web.
3. **On ajoute un jeton spécial, appelé CLS**. Il ne vient d'aucun lab : son rôle est de collecter l'information de tous les autres, comme le secrétaire d'une réunion qui écrit le compte-rendu.
4. **Les jetons « discutent » entre eux** grâce au mécanisme d'**attention** (2 couches, 4 « têtes » qui regardent chacune des choses différentes).
5. **Le masque** dit au Transformer d'ignorer les labs non faits. Ils ne participent pas à la discussion.
6. À la fin, on garde seulement le jeton CLS : c'est le **résumé des labs** (64 nombres).

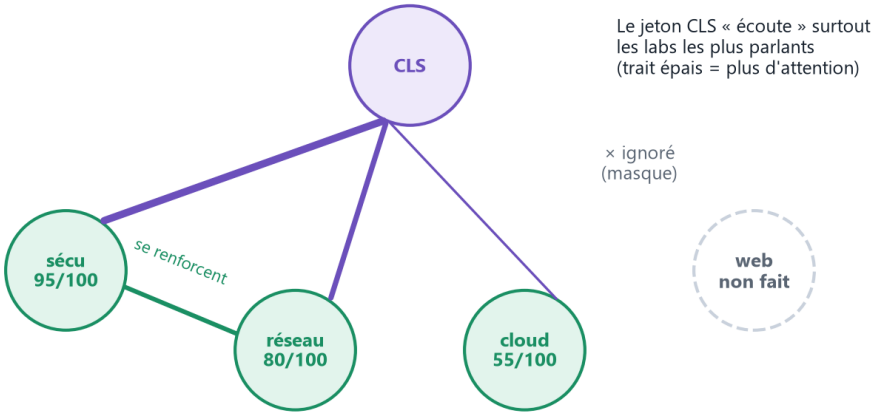


Figure 6. L'idée de l'attention. Le jeton CLS écoute davantage les labs qui en disent le plus sur l'étudiant. Les labs peuvent aussi se renforcer entre eux : bon en sécurité et bon en réseau, c'est un signe fort de cybersécurité. Le lab non fait est ignoré.

La décision finale

Les deux résumés sont collés ensemble (64 + 64 = 128 nombres), puis passent dans un dernier petit MLP qui sort **9 scores**, un par métier. Une fonction appelée **softmax** transforme ces scores en **pourcentages** dont la somme fait 100 %. On affiche les 3 plus grands.

> **Pourquoi un Transformer pour les labs, et pas un simple MLP ?**
> • **Labs manquants** : chaque étudiant fait des labs différents. Le masque les ignore proprement, sans inventer de notes.
> • **Combinaisons** : l'attention repère les associations entre labs (sécurité + réseau → cybersécurité).
> • **Facile à agrandir** : ajouter un nouveau lab = ajouter un jeton, sans tout reconstruire.
> • **Explicable** : on peut regarder quels labs le jeton CLS a le plus écoutés.

Comment le modèle apprend

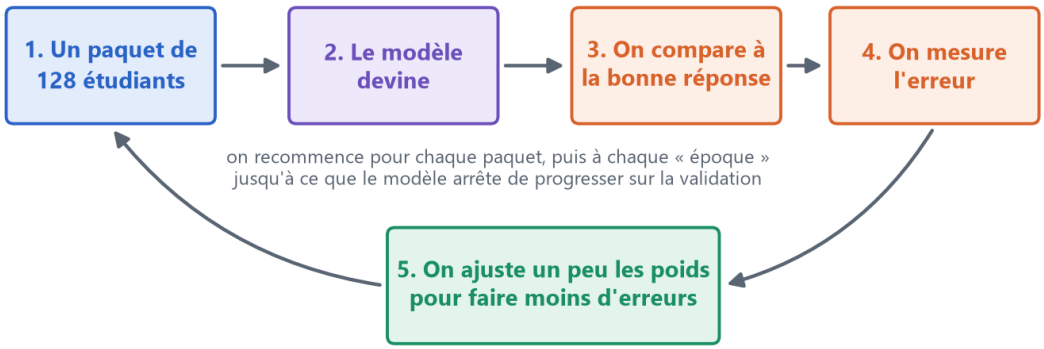


Figure 7. La boucle d'apprentissage. Un passage complet sur les 4 000 étudiants s'appelle une « époque » (32 paquets de 128).

Les « astuces » utilisées pendant l'entraînement :

Technique	Explication simple
Erreur pondérée	Se tromper sur un métier rare coûte plus cher (poids de classes).
Label smoothing	On dit au modèle « c'est presque sûr » au lieu de « c'est 100 % sûr », pour qu'il reste modeste.
Lab dropout (15 %)	À chaque paquet, on cache au hasard 15 % des labs faits. Le modèle apprend ainsi à se débrouiller quand il manque des informations.
Dropout	On éteint au hasard quelques neurones pendant l'entraînement, pour éviter que le modèle apprenne par cœur au lieu de comprendre.
AdamW	La méthode qui ajuste les poids. Elle freine aussi les poids trop grands.
Warm-up + cosinus	On commence à apprendre doucement, puis plus vite, puis de plus en plus finement à la fin.
Gradient clipping	On limite la taille de chaque correction, pour éviter un « coup de volant » trop brusque.
Early stopping	Après chaque époque, on regarde la note sur la validation. Si elle ne progresse plus pendant 25 époques, on arrête et on garde la meilleure version. Ici : arrêt à l'époque 45.

Les modèles comparés

Pour prouver que CareerNet est utile, on l'a comparé à 4 autres modèles, entraînés sur les mêmes données :

Modèle	Type	Idée
Régression logistique	machine learning classique	le plus simple : une seule formule par métier
Gradient Boosting	machine learning classique	des centaines de petits arbres de décision qui se corrigent entre eux
MLP	deep learning	un seul réseau simple qui lit tout d'un coup, sans Transformer

Modèle	Type	Idée
CareerNet + labels doux	deep learning	CareerNet qui apprend aussi un « degré d'adéquation » à chaque métier
CareerNet	deep learning	notre modèle, décrit dans ce chapitre

CRISP-DM · Phase 5

Évaluer : les résultats

Tous les modèles passent le **même examen** : les 500 étudiants du test, jamais vus pendant l'entraînement. Les modèles de deep learning sont entraînés **3 fois** avec des départs au hasard différents, et on donne la moyenne. Le « ± » indique de combien le résultat varie d'une fois à l'autre.

1. Qui est le meilleur modèle ?

Modèle	Accuracy	Macro-F1	Top-3
Régression logistique	0,896	0,888	0,988
Gradient Boosting	0,892	0,875	0,986
MLP	0,898 ± 0,005	0,890 ± 0,006	0,982 ± 0,003
CareerNet (modèle retenu)	**0,909 ± 0,003**	**0,903 ± 0,007**	**0,989 ± 0,002**
CareerNet + labels doux	0,905 ± 0,013	0,896 ± 0,014	0,995 ± 0,001

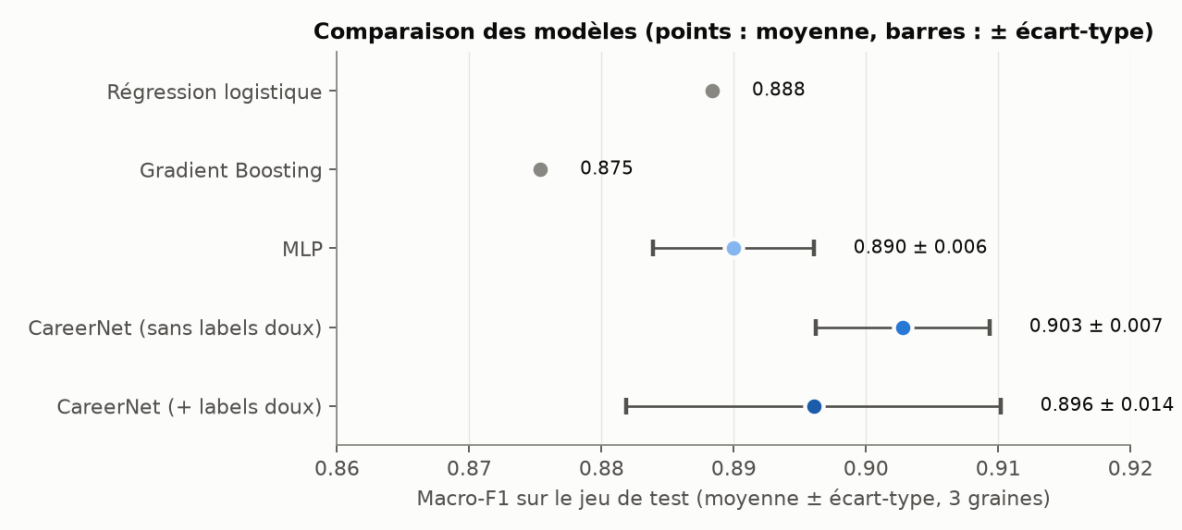


Figure 8. Macro-F1 de chaque modèle. Le point est la moyenne, la barre montre la variation entre les 3 entraînements.

- **CareerNet gagne** : +1,5 point de macro-F1 par rapport à la régression logistique, +1,3 point par rapport au MLP. C'est une avance réelle mais modeste : les données contiennent volontairement des erreurs, donc aucun modèle ne peut atteindre 100 %.
- **Les labels doux n'aident pas** : résultat moyen un peu moins bon et deux fois plus instable. Et ils n'existeraient pas avec de vrais étudiants. On les abandonne.
- **Top-3 à 98,9 %** : afficher 3 propositions est très fiable.

2. Résultats métier par métier

La **précision** répond à « quand le modèle dit ce métier, a-t-il raison ? ». Le **rappel** répond à « parmi les vrais étudiants de ce métier, combien le modèle en retrouve ? ». Le **F1** combine les deux.

Métier	Précision	Rappel	F1	Étudiants	Avis
Data Scientist	0,971	0,971	0,971	68	très bon

Métier	Précision	Rappel	F1	Étudiants	Avis
Frontend/Web Developer	0,946	0,972	0,959	72	très bon
Database Administrator	0,980	0,906	0,941	53	très bon
QA/Test Engineer	0,912	0,912	0,912	57	bon
Cybersecurity Analyst	0,879	0,927	0,903	55	bon
DevOps Engineer	0,852	0,958	0,902	24	bon, malgré sa rareté
System/Network Admin	0,848	0,918	0,882	61	moyen
Backend Developer	0,891	0,845	0,867	58	moyen
Cloud Engineer	0,870	0,769	0,816	52	point faible

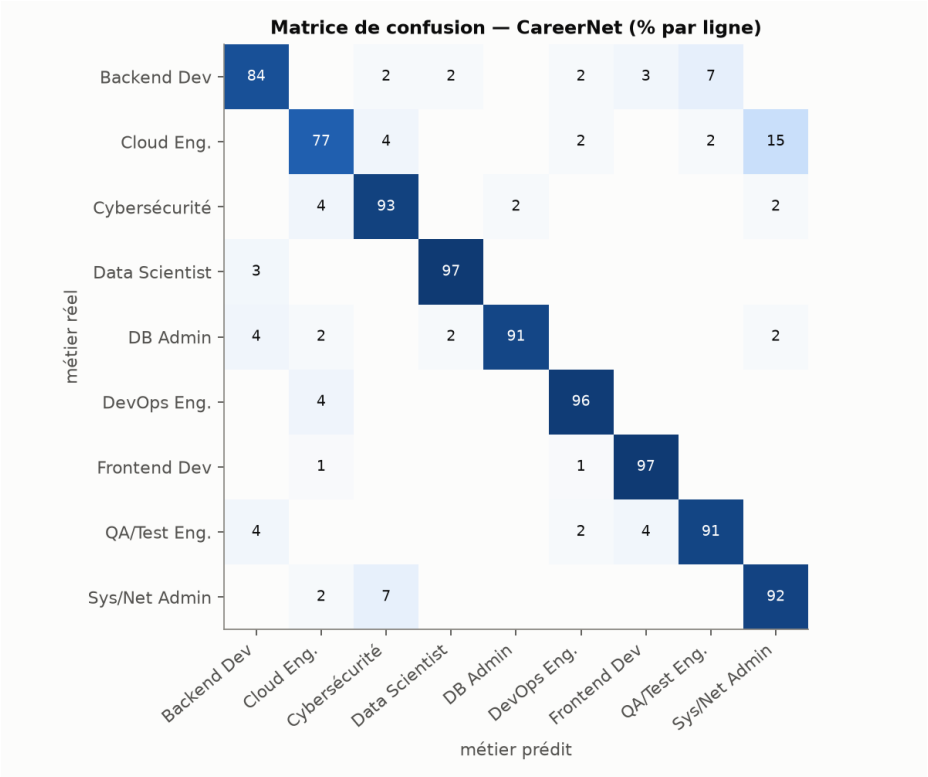


Figure 9. Matrice de confusion. Chaque ligne est le vrai métier, chaque colonne le métier prédit. La diagonale = les bonnes réponses. Les cases hors diagonale = les confusions.

Les confusions les plus fréquentes concernent des métiers qui se ressemblent : **8 Cloud Engineers** pris pour des System/Network Admins (tous deux travaillent sur le réseau et les serveurs), **4 System/Network Admins** pris pour des Cybersecurity Analysts, et **4 Backend Developers** pris pour des QA/Test Engineers (tous deux programment).

3. L'apprentissage s'est-il bien passé ?

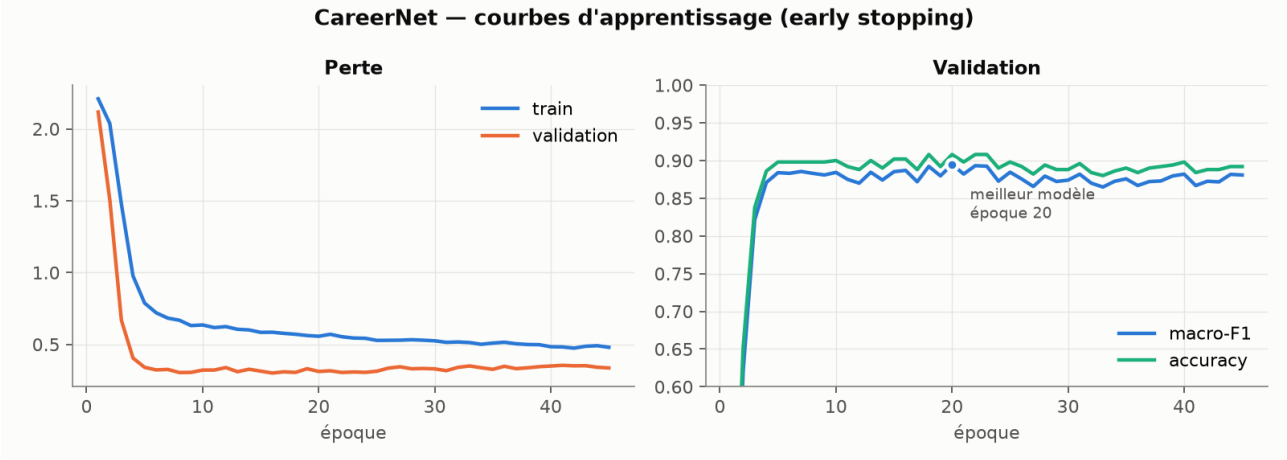


Figure 10. À gauche : l'erreur diminue au fil des époques. À droite : la note sur la validation monte puis se stabilise ; le meilleur modèle est gardé.

L'erreur sur l'entraînement est plus haute que sur la validation. Ce n'est pas un problème : pendant l'entraînement, on rend volontairement la tâche plus difficile (dropout, labs cachés), ce qu'on ne fait pas pendant la validation. Les deux courbes ne s'écartent pas : le modèle n'apprend pas « par cœur » (pas de surapprentissage).

4. Et si l'étudiant a fait peu de labs ?

On a refait l'examen en ne gardant que 0, 1, 2… labs par étudiant, choisis au hasard.

Modèle	0 lab	1	2	3	4	5	tous
Gradient Boosting	0,736	0,745	0,768	0,781	0,812	0,842	0,875
MLP	0,773	0,809	0,798	0,823	0,836	0,858	0,886
CareerNet	**0,734**	**0,771**	**0,791**	**0,812**	**0,842**	**0,884**	**0,906**

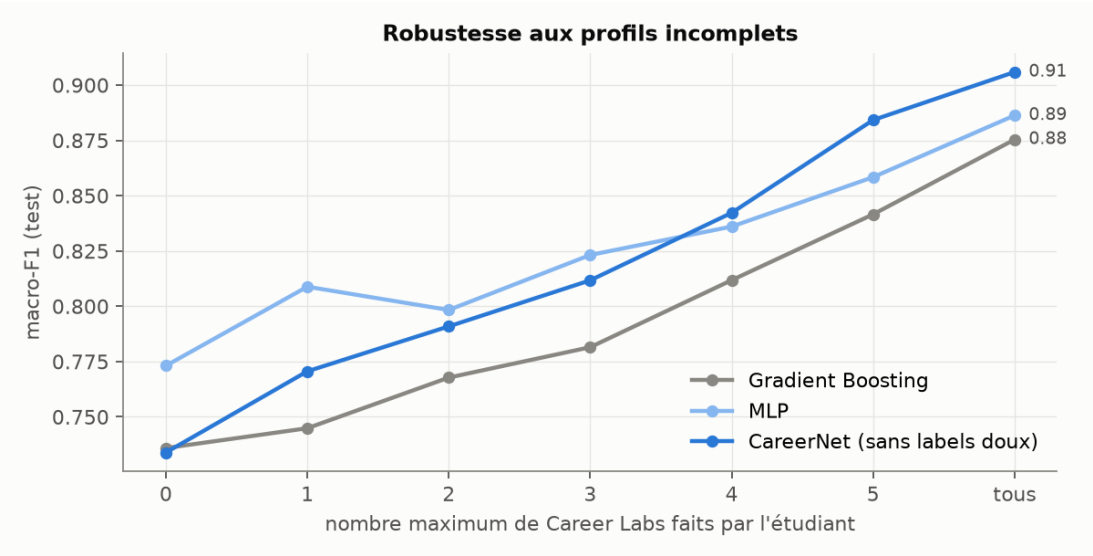


Figure 11. Macro-F1 selon le nombre de labs faits. Plus l'étudiant fait de labs, meilleure est la recommandation.

- **Les labs sont très utiles** : CareerNet passe de 0,734 sans lab à 0,906 avec tous les labs, soit **+17 points**. C'est la preuve que les Career Labs ont de la valeur pour la plateforme.
- Avec **4 labs ou plus**, CareerNet est le meilleur. Avec moins de 4 labs, le MLP fait un peu mieux : c'est une piste d'amélioration.

5. Quelles informations comptent le plus ?

Pour le savoir, on **mélange** une information entre les étudiants (par exemple, on donne à chacun les auto-évaluations d'un autre) et on regarde de combien la note baisse. Plus elle baisse, plus l'information était importante.

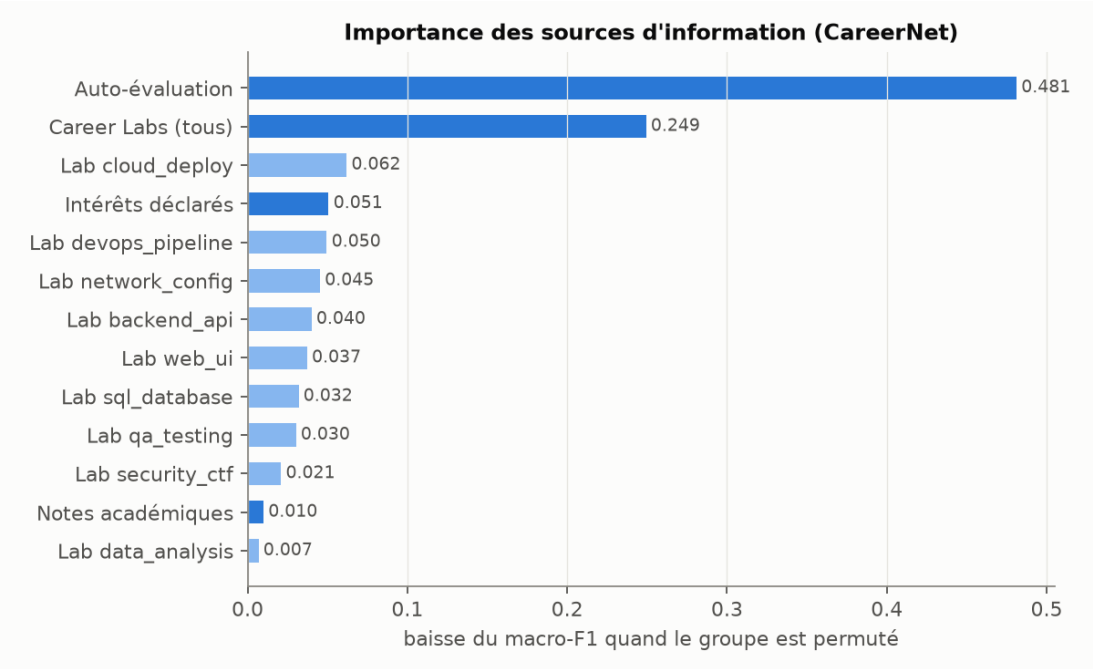


Figure 12. Baisse du macro-F1 quand on mélange chaque information.

> **À retenir avec prudence**
> Les **auto-évaluations** comptent le plus (baisse de 0,48), devant les **labs** (0,25). Les notes scolaires comptent très peu (0,01).
> C'est sans doute un effet de la simulation : les auto-évaluations y sont une copie un peu bruitée des vraies aptitudes. Dans la vraie vie, les gens se jugent moins bien, et les labs devraient compter davantage.

6. Le modèle est-il juste ?

Genre	Étudiants au test	Accuracy	Macro-F1
Femmes	231	0,905	0,898
Hommes	269	0,914	0,911

La différence est de 1 point. Un calcul statistique (bootstrap) donne une marge d'incertitude de −6 à +4 points : elle contient 0, donc la différence **n'est pas significative**, elle peut venir du hasard. Rappel : le genre n'est jamais donné au modèle.

> **En bref**
> CareerNet est le meilleur des 5 modèles (macro-F1 0,903, top-3 98,9 %). Il est juste entre femmes et hommes. Les labs apportent 17 points. Son point faible : Cloud Engineer, confondu avec System/Network Admin.

CRISP-DM · Phase 6

Déployer : utiliser le modèle

Une fois entraîné, le modèle est enregistré dans un fichier. Un petit programme, **predict.py**, permet de l'utiliser pour n'importe quel nouvel étudiant. Il est prêt à être branché sur le site de la plateforme.

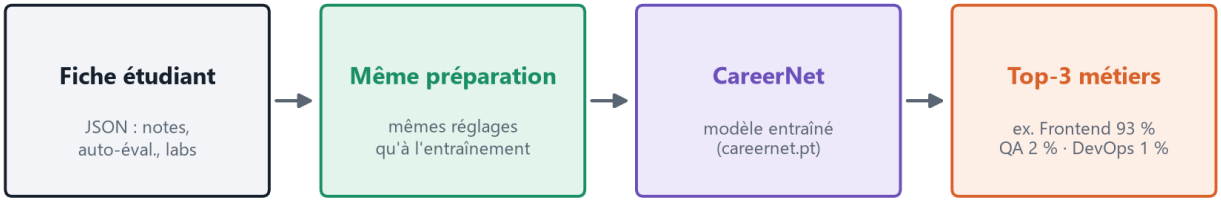


Figure 13. Ce qui se passe quand un nouvel étudiant demande une recommandation.

1. La plateforme envoie la fiche de l'étudiant : notes, auto-évaluations, intérêts, et les résultats des labs qu'il a faits.
2. Le programme applique **exactement la même préparation** qu'à l'entraînement, avec les mêmes réglages enregistrés. C'est indispensable : sinon le modèle recevrait des nombres sur une autre échelle.
3. Un lab n'est compté comme fait que si ses 5 mesures sont présentes. Sinon, il est masqué.
4. CareerNet calcule les 9 pourcentages et renvoie les 3 meilleurs.

Exemple de réponse pour un étudiant orienté web :

```
Frontend/Web Developer   93,5 %
QA/Test Engineer            2,0 %
DevOps Engineer             1,3 %
labs faits : 3
```

Les fichiers nécessaires

Fichier	Rôle
careernet.pt	les 88 521 poids appris par le modèle
preprocess.json	les réglages de préparation (moyennes, écarts-types, ordre des colonnes, liste des métiers)
predict.py	le programme qui fait la prédiction

Cas testés : quand le modèle réussit et quand il se trompe

Pour vérifier le comportement du modèle, on lui a donné 8 étudiants très différents, dont certains pièges.

Cas	Ce qu'on donne au modèle	Labs	Réponse n°1	Verdict
Profil data clair	Maths 17,5/20, Python 9/10, intérêt data 5/5, lab data à 92/100	3	Data Scientist 96,5 %	**correct**
Profil web clair	Web 9/10, intérêt web design 5/5, lab web à 95/100	3	Frontend 93,5 %	**correct**
Profil sécurité clair	Sécurité 9/10, Linux 9/10, lab CTF à 96/100	2	Cybersécurité 98,8 %	**correct**
Profil mixte	Bon partout (6 à 10), intérêt sécurité 5/5, CTF à 100/100	6	Cybersécurité 96,2 %	**cohérent**
Aucun lab	Profil sécurité, mais aucun lab fait	0	Cybersécurité 95,6 %	**fonctionne**
Profil ambigu	Cloud, Docker et Linux à 8/10, labs cloud et DevOps à environ 80	2	DevOps 96,3 % (Cloud 3,5 %)	**trop sûr de lui**
Signaux contraires	Se dit fort en sécurité, mais son seul lab est web, réussi à 97/100	1	Cybersécurité 92,7 %	**ignore le lab**
Information manquante	On retire l'auto-évaluation en Python	–	le programme s'arrête avec une erreur	**à corriger**

Ce qu'on apprend de ces cas

- **Les profils clairs** sont très bien reconnus, même avec seulement 2 ou 3 labs.
- **Sans aucun lab**, le modèle marche encore : le jeton CLS existe toujours, et la décision repose alors sur le profil.
- **Profil ambigu** : la bonne réaction serait d'hésiter entre Cloud et DevOps (par exemple 55 % / 40 %). Le modèle est **trop sûr de lui**. On peut corriger cela avec une technique appelée *calibration*. En attendant, montrer le top-3 limite le risque.
- **Signaux contraires** : le modèle fait plus confiance à ce que l'étudiant dit de lui-même qu'à son excellent résultat pratique. C'est la conséquence du poids trop fort des auto-évaluations (phase 5).
- **Information manquante** : le programme ne vérifie pas que la fiche est complète. Il faut ajouter une vérification qui renvoie un message clair.

Limites et améliorations

Les limites à connaître

1. **Les données sont simulées.** Les résultats montrent que le modèle sait retrouver la logique de la simulation. Il faudra les confirmer avec de vrais étudiants.
2. **Les profils métiers sont à moitié des avis d'experts**, pas seulement des données O*NET.
3. **Les auto-évaluations comptent sans doute trop**, à cause de la façon dont elles ont été simulées.
4. **Le modèle est parfois trop sûr de lui** sur les profils entre deux métiers.
5. **L'examen contient 500 étudiants**, dont seulement 24 DevOps : les notes par métier ont une marge d'erreur de quelques points.

Les améliorations proposées

Amélioration	Ce que ça corrige
Collecter de vraies données sur la plateforme, puis réentraîner le modèle dessus (fine-tuning)	données simulées
Calibrer les pourcentages (temperature scaling)	modèle trop sûr de lui
Cacher plus de labs pendant l'entraînement (lab dropout plus fort)	moins bon que le MLP avec 0 à 3 labs
Montrer à l'étudiant quels labs ont le plus compté (poids d'attention)	explication des recommandations
Vérifier la fiche de l'étudiant avant la prédiction	erreur si une information manque
Validation croisée (refaire l'examen sur plusieurs découpages)	marge d'erreur des résultats

Conclusion

En suivant la méthode CRISP-DM, on a construit un modèle de recommandation complet : des profils métiers réels, des données d'étudiants réalistes, une préparation sans fuite, une architecture adaptée aux labs manquants, et une évaluation honnête.
CareerNet est le meilleur des modèles testés, propose le bon métier dans son top-3 pour 98,9 % des étudiants, et montre que les Career Labs améliorent nettement la recommandation.

Petit dictionnaire des mots techniques

Mot	Signification
Accuracy	Pourcentage de bonnes réponses.
Attention	Mécanisme du Transformer : chaque jeton décide à quels autres jetons il « fait attention ».
Batch (paquet)	Groupe d'exemples traités ensemble pendant l'entraînement (ici 128 étudiants).
Classe	Une des réponses possibles. Ici, un des 9 métiers.
CLS	Jeton spécial qui résume toute la séquence. Idée reprise du modèle BERT.
Dropout	Éteindre au hasard des neurones pendant l'entraînement pour éviter le par-cœur.
Early stopping	Arrêter l'entraînement quand la note de validation ne progresse plus.
Embedding	Représentation d'une chose (ici, l'identité d'un lab) par une liste de nombres apprise.
Époque	Un passage complet sur tous les exemples d'entraînement.
F1	Note qui combine précision et rappel. Macro-F1 = moyenne des F1 de tous les métiers.
Fuite de données	Erreur où le modèle voit, sans le vouloir, des informations de l'examen.
Jeton (token)	Une unité lue par le Transformer. Ici, un lab.
Masque	Liste qui indique quels labs ignorer parce qu'ils n'ont pas été faits.
MLP	Perceptron multicouche : des couches de neurones empilées. Le réseau le plus simple.
Normalisation	Mettre toutes les informations à la même échelle, centrées autour de 0.
O*NET	Base de données publique américaine qui décrit plus de 900 métiers.
Poids	Les réglages internes du réseau, ajustés pendant l'apprentissage.
Précision / Rappel	Précision : quand le modèle dit X, a-t-il raison ? Rappel : retrouve-t-il tous les vrais X ?
Softmax	Fonction qui transforme des scores en pourcentages dont la somme fait 100 %.
Stratifié	Découpage qui garde la même proportion de chaque métier dans chaque groupe.
Surapprentissage	Quand le modèle apprend par cœur l'entraînement et devient mauvais sur de nouveaux cas.
Top-3 accuracy	Pourcentage d'étudiants pour qui le bon métier est dans les 3 propositions.
Transformer	Type de réseau basé sur l'attention, utilisé dans les grands modèles de langage.

Pour refaire toutes les expériences

Il suffit de lancer, dans l'ordre : step1_collect_onet.py, step2_generate_students.py, preprocess.py, train.py, experiments.py, puis predict.py pour tester un étudiant. L'entraînement prend environ 1 minute sur un ordinateur normal, et toutes les expériences environ 10 minutes.